

VOCATIONAL TRAINING PROJECT REPORT

ON

EDUVORA

A Full-Stack Course Access, Referral, Payment and KYC Management Platform

Submitted by

Indrasish Biswas

Roll / Registration No.: 24E102B91

B.Tech CSE

Submitted To

Subham Chakraborty

CEO & Founder

WebMetrix

Web Development Project Category

CERTIFICATE OF SUBMISSION

This is to certify that the project report entitled **EDUVORA**, a full-stack web development project for course-package access, referral-network management, payments and KYC processing, has been prepared and submitted by **Indrasish Biswas** (24E102B91), B.Tech CSE, for vocational training/project evaluation at WebMetrix.

The report documents the implementation available in the EDUVORA codebase, including its React frontend, Node.js backend, SQL Server stored-procedure data layer, payment workflow and asynchronous KYC worker. It is intended as a technical submission; validation of organisational approval and signatures remains with the submitting institution and reviewer.

Submitted to

Subham Chakraborty
CEO & Founder, WebMetrix

ABSTRACT

EDUVORA is a full-stack web application that brings together packaged learning access, referral-network visibility, user profile management, payment initiation and identity-verification workflows. The application is designed around a React single-page frontend and a Node.js/Express REST backend. It persists operational data through SQL Server stored procedures and presents authenticated users with dashboard, profile, referral, network, course, order, earnings and KYC views.

The implemented KYC pathway is the project's most distinct end-to-end asynchronous workflow. After a protected multipart submission, the Node.js backend applies server-side format and file checks, saves documents in a temporary repository, records the submission through a stored procedure, and publishes a persistent Celery-compatible task to RabbitMQ. A Python worker then checks image dimensions, blur and exposure, masks Aadhaar QR and number regions where applicable, moves accepted files to permanent storage, updates the SQL Server status, and calls a backend webhook. The platform also integrates Google sign-in, SMTP-driven email events, an IFSC lookup and Cashfree payment-order creation with payment-status processing through a webhook.

This report is intentionally evidence-based. It differentiates live backend-connected modules from frontend demonstration areas: the current order list, earnings panels and course-player curriculum contain static/mock presentation data, and the backend does not expose corresponding live endpoints in this codebase. This distinction prevents the report from overstating present functionality.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Subham Chakraborty, CEO & Founder of WebMetrix, for the opportunity and guidance associated with this project. I am thankful for the practical exposure to full-stack web development, payment integration, database-driven workflows and document-processing architecture that shaped EDUVORA. I also acknowledge the support of peers and faculty whose feedback helped refine the implementation and documentation.

TABLE OF CONTENTS

Section	Page
1. Introduction and project description	6
2. System study and requirements	8
3. Software requirement specification	10
4. System design and architecture	12
5. Database design	15
6. Implementation modules	18
7. Testing and verification	22
8. Application walkthrough and screenshot placeholders	24
9. Challenges and limitations	34
10. Conclusion, future scope and bibliography	36

Page references are based on the generated report layout; screenshot placeholders reserve space for final UI captures to be inserted manually.

1. INTRODUCTION

1.1 Project Description

EDUVORA is a web development project that combines customer-facing account management with the operational workflows required by a package-based learning and referral environment. Users can register or authenticate, complete personal and banking information, receive and share referral links, inspect a visualised downline network, browse packages, initiate package payments, and submit KYC documents. The backend is structured as REST routes and controllers that invoke SQL Server stored procedures; this means the core business entities and transactions are retained in the database tier rather than in JavaScript ORM models.

1.2 Problem Statement

Managing registrations, profile completeness, referral relationships, paid package entitlement and identity documents across separate manual systems creates inconsistent status visibility and makes compliance workflows difficult to trace. EDUVORA addresses this by providing a single authenticated interface and a backend that records the underlying user, payment, referral and KYC actions through named database procedures.

1.3 Verified Objectives

No.	Objective	Implementation evidence
01	Secure account access	Standard registration/login, Google credential verification, bcrypt password hashes and signed JWTs.
02	Profile and referral onboarding	Profile APIs, availability check, lookup endpoints and referral assignment/list procedures.
03	Package purchase initiation	Active-package query plus Cashfree order creation and webhook processing.
04	Asynchronous KYC processing	Multer upload, SQL persistence, RabbitMQ/Celery queue, Python checks and webhook status update.

2. SYSTEM STUDY

2.1 Existing and Proposed System

A fragmented workflow would require separate registration records, informal referral tracking, manual document review and unconnected payment reconciliation. EDUVORA proposes a unified browser application with a common authenticated session and central SQL Server data layer. The referral and network views are driven by dedicated stored-procedure outputs, and KYC documents are handled in a controlled temporary-to-permanent storage path rather than remaining only in a browser upload.

2.2 Hardware and Software Requirements

Layer	Verified requirement / technology	Purpose
Client	Modern browser; JavaScript enabled	React SPA and Cashfree JavaScript checkout redirect flow.
Frontend	React 19, Vite 8, React Router, Redux Toolkit, Tailwind CSS	SPA, routing, shared state and visual interface.
Backend	Node.js, Express 5, mssql	REST routing and stored-procedure execution.
Database	Microsoft SQL Server	User, profile, package, order, wallet, referral and KYC data.
Async worker	Python, Celery, RabbitMQ, OpenCV, Tesseract, SQLAlchemy	Queued KYC quality, masking, storage and status updates.
Integrations	Cashfree, Google OAuth, SMTP, IFSC endpoint	Payments, sign-in, notifications and bank metadata.

2.3 Scope Boundary

The codebase supplies no automated test suite and no deployment manifest. The report therefore records build verification and endpoint/module inspection rather than inventing production performance figures or deployment claims.

3. SOFTWARE REQUIREMENT SPECIFICATION

3.1 Users and Access Model

The standard client user is assigned RoleID 3 on registration. A separate frontend route exists for a super-admin creation interface, but the route itself is not protected by the application's ProtectedRoute wrapper; this report therefore does not claim a verified server-side administrator authorization model. Protected API endpoints use the isLoggedIn middleware, which accepts a Bearer token or cookie, verifies a JWT and places its user identity on the request.

3.2 Functional Requirements

- Register with full name, email, optional mobile number, password and optionally a referral code; reject duplicate or invalid referral conditions through the database procedure.
- Log in with credentials or a Google ID token; use the returned session information in the browser session storage and Axios authorization interceptor.
- Read and edit profile data, upload PNG/JPG profile images, and retrieve lookup data for gender, state, city and account type.
- Read active packages and initiate a Cashfree order for an authenticated user with required contact details.
- Submit a PAN card and selected identity document for KYC, observe KYC status, and process the request asynchronously.
- Retrieve a referral list with server-procedure search/date filtering and retrieve network-tree and chart payloads.

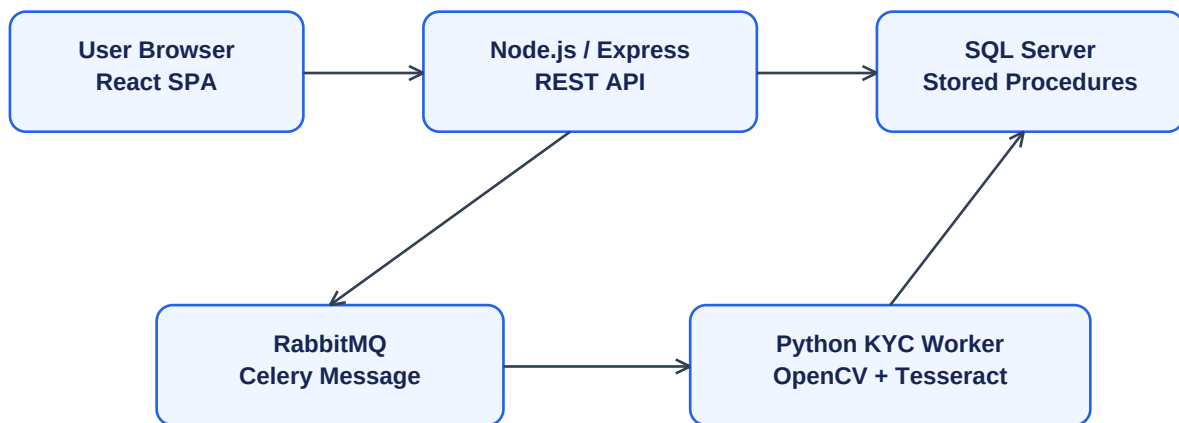
3.3 Non-functional Considerations

The app uses JSON HTTP APIs, a responsive React component structure, localisation keys shared by frontend and backend, structured logging and environment variables for service configuration. Actual SLA, load, uptime and accessibility measurements are not present in the repository.

4. SYSTEM DESIGN

4.1 System Perspective

The application divides responsibilities across a browser client, Express API, SQL Server database, third-party payment/auth/email services, RabbitMQ, and a Python Celery worker. SQL stored procedures form the principal persistence/business-transaction boundary. The frontend uses Redux slices for authentication, profile, network, referral, payment and KYC state; Axios centralises base URL, credentials and authorization-header behaviour.



Synchronous requests use the REST API; KYC tasks are published to the Celery queue and results update SQL Server and the webhook.

Figure 1: Verified EDUVORA architecture and KYC asynchronous path.

4.2 Authentication and Session Flow

During standard registration, the backend creates a UUID, derives a user ID and unique username, hashes the password with bcrypt (salt rounds configured as 10), and signs a one-day JWT. The database procedure performs duplicate/referral validation. Login compares the stored hash, generates a JWT and session identifier, and updates session records through a procedure. Google login verifies the supplied ID token with google-auth-library before using the corresponding database workflow. The frontend Axios interceptor sends a session identifier as a Bearer token when present and redirects after a 401 response.

4. SYSTEM DESIGN CONTINUED

4.3 KYC Activity Flow

Step	Actual workflow
1. Collect	KycVerificationContent combines personal details, document choice, upload controls, review/consent and status components.
2. Validate	Frontend rules are mirrored by kycBasicValidator on the server. Multer allows JPEG, PNG and PDF MIME types at the route.
3. Temporarily store	The KYC route looks up the TempKYC repository path from SQL Server and creates a UUID-specific directory; files are standardised by type.
4. Persist and queue	EV_ManageUserKYC records the submission. The Node utility posts a persistent JSON Celery task to the RabbitMQ management API.
5. Process	Celery checks image resolution, Laplacian-variance blur and dark/light pixel ratios. Aadhaar images additionally undergo QR and OCR-based masking.
6. Finalise	Accepted files move from TempKYC to KYC storage; the worker updates KYC status in SQL Server and posts the result to the Express webhook. Rejected files are deleted from temporary storage.

4.4 Payment Activity Flow

The protected create-order controller first requests a generated order number through EV_ProcessCashfreePayment with the INITIATE action. It then creates a Cashfree order using the authenticated user UUID and the user's name, email and mobile data. The Cashfree webhook accepts gateway event data and invokes the same procedure with WEBHOOK. According to brain.md, successful processing updates order/payment states and inserts an active user-package record. Webhook signature verification is not implemented in the inspected controller, so it is not claimed as a protection.

5. DATABASE DESIGN

EDUVORA uses Microsoft SQL Server via the Node mssql library and Python SQLAlchemy/pyodbc. The project documents its schema in backend/brain.md and uses stored procedures from its controllers. The following table summarises the principal entities relevant to implemented user-facing flows.

Entity group	Purpose and relationship
Tb_User and Tb_UserDesc	Core account and extended profile information, joined by UUID. Tb_BankInfo supplies bank fields.
Tb_Referral and Tb_UserRelationship	Sponsor/new-user assignment and referral hierarchy used by the network and referral procedures.
Tb_Package, Tb_Courses, Tb_Module, Tb_Lesson	Package and learning-content catalogue. The schema defines modules and lessons; current course-player UI uses mock curriculum data.
Tb_Order, Tb_OrderItem, Tb_Payment, Tb_Invoice	Purchase and payment lifecycle; order items link to packages and payment data retains gateway response.
Tb_UserPackage	Entitlement mapping inserted after a successful payment according to EV_ProcessCashfreePayment.
Tb_Wallet, Tb_WalletTransaction, Tb_CommissionLedger	Commission/wallet accounting model; current frontend earnings widgets are presentational sample data in this codebase.
Tb_UserKYC and Tb_KYCStatusMaster	One KYC record per UUID, document paths, status and rejection reason.
Master tables	Roles, ranks, package status, order/payment statuses, account types, gender, city, commission rules and event templates.

5.1 Important Constraints and Data Handling

The KYC schema declares UUID unique per user. The registration procedure reports collisions for account identifiers and invalid/missing referral assignments. File repository locations are obtained from Tb_FileRepositoryMaster through EV_GetFileRepositoryPath. Sensitive credentials, connection-string values, JWT secrets, payment secrets and service passwords are excluded from this report.

5. DATABASE DESIGN CONTINUED

5.2 Stored Procedure Integration

Procedure	Controller use
EV_CreateUser / EV_LogIn / EV_UpdateUserSession	Registration, credential lookup and session update.
EV_GetUserProfile / EV_EditUser / EV_UpdateProfilePicture / EV_UpdateAboutMe	Profile retrieval and updates.
EV_AssignReferral / EV_GetMyReferralsList / EV_GetMyNetwork	Referral relationship, list and hierarchy/analytics retrieval.
EV_GetPackages	Active package retrieval.
EV_ProcessCashfreePayment	Initial pending order/payment records and webhook-driven payment transition.
EV_ManageUserKYC	KYC retrieval, submission and worker status update.
EV_GetFileRepositoryPath	Profile and KYC file-storage root lookup.
EV_GetEmailTemplate	Email service template lookup.

5.3 Commission Calculation Boundary

The schema includes Tb_CommissionRuleMaster fields for commission percentage and flat amount, but the inspected Node and Python application code does not calculate commissions. The report therefore does not state a commission formula or claim that UI earnings values are calculated live. Any computation is database-side or outside this repository's exposed application code.

6. IMPLEMENTATION MODULES

6.1 Authentication and Recovery

Frontend login, registration and forgot-password pages use Redux async thunks. The backend exposes /auth/register, /auth/login, /auth/logout and /auth/google, plus OTP send/verify/resend and password reset routes. OTP routines use an email-existence check, generated numeric OTP, expiry configuration and SMTP email templates. Password reset hashes the new password before invoking its database procedure. User-facing notifications are delivered with react-toastify.

6.2 Profile and File Management

Profile pages comprise account overview, personal/contact/address/bank cards, account security and About Me editing. The profile route provides lookup endpoints and an IFSC verification endpoint. Profile image uploads are restricted in the route to PNG/JPG and written under a user UUID directory using the repository path returned from SQL Server. Existing display-picture files are removed before replacement.

6.3 Package and Payment Module

The dashboard fetches packages through the backend and uses package cards/modal components. The payment controller uses Cashfree SDK server-side order creation. The frontend includes Cashfree JavaScript dependencies and payment success/failure routes. The inspected order-history page currently contains a local array and Excel export function; it is therefore a UI prototype rather than evidence of a live order-history API.

6. IMPLEMENTATION MODULES CONTINUED

6.4 Referral and Network Module

After protected signup-related referral assignment, the backend can retrieve a direct/indirect referral list and an up-to-level-2 network payload. Network UI components render team dashboard cards, package distribution and registration-trend charts, tree controls, zoom/pan support and a genealogy tree. Referral UI includes a profile card, shareable referral link, QR code, statistics, rewards/tips and data table. Server-side procedure documentation specifies search over user ID, name and registration date and filters for referral data.

6.5 Course and Earnings Presentation

The frontend exposes My Courses, Course Player, earnings summary/history/wallet and withdrawal modal components. The CoursePlayer file declares mockCourseData, and the earnings/order component group provides formatted demonstration cards, tables and charts without inspected corresponding backend routes. These pages are valuable interface work, but their values and curriculum should not be interpreted as confirmed live SQL data.

6.6 KYC Module

The KYC form selects the identity proof type and presents dynamic document requirements from rules configuration. Basic server validation covers personal names, PAN, supported proof formats and file presence. The Python worker's quality decision can be represented as: $\text{pass} = (\text{width} \geq \text{minimum}) \text{ AND } (\text{height} \geq \text{minimum}) \text{ AND } (\text{Laplacian variance} \geq \text{minimum}) \text{ AND } (\text{dark-pixel ratio} \leq \text{maximum}) \text{ AND } (\text{light-pixel ratio} \leq \text{maximum})$. Thresholds are loaded from rules.json; this report intentionally does not invent data-quality accuracy metrics.

7. SECURITY AND VALIDATION

Area	Verified implementation
Passwords	bcrypt hashing before registration/password-reset stored-procedure calls.
Session authentication	JWT verification middleware reads Bearer token or cookie, checks signature and exposes decoded identity.
Google authentication	Google OAuth client verifies ID token with configured client ID.
Authorization	Protected API routes are used for profile retrieval/editing, referral assignment/listing, network, payment order creation and KYC.
Input/data validation	Database procedure return codes; KYC basic validator; client-side rule configuration; multer MIME filters.
Sensitive document protection	Aadhaar OCR/QR masking in worker; encryption utility exists for AES-256-CBC encryption/decryption of referral-related user ID values.
Configuration	Environment variables configure database, JWT, OAuth, SMTP, payment, RabbitMQ and service values without hard-coding reportable secrets.
Logging/errors	Winston logger and controller error handling are used throughout key workflows.

Security limitations observed in code are not disguised: the payment webhook controller does not verify a Cashfree signature, the KYC webhook has no inspected service-to-service authentication, and frontend super-admin routing is not wrapped in the standard protection component. These are identified implementation boundaries, not claims of secure production deployment.

8. SOFTWARE TESTING AND VERIFICATION

No automated test folder or unit/integration test script is present in the inspected repository. Verification was therefore performed at code and build level, supplemented by a successful production frontend build. The following test cases are suitable for operational validation; results marked “verified by inspection” reflect observable route/controller/component wiring, not a live production test execution.

Test case	Expected result	Status
Frontend production build	Vite transforms and emits production assets.	Passed in workspace
Unauthenticated protected API request	JWT middleware should reject absent/invalid token.	Verified by inspection
KYC unsupported file	Multer route rejects MIME types outside JPEG/PNG/PDF.	Verified by inspection
Blurred/low-quality KYC image	Worker rejects if quality threshold fails and deletes temporary folder.	Verified by inspection
Aadhaar masking failure	Worker rejects if OCR cannot identify sufficient digit groups.	Verified by inspection
Successful KYC task	Worker moves files, updates KYC state and posts webhook.	Verified by inspection
Cashfree order initiation	Controller validates contact details, calls procedure then payment SDK.	Verified by inspection
Payment event processing	Webhook maps success/failed state into payment procedure.	Verified by inspection

8.1 Build Observation

The frontend build completed successfully. The bundler emitted a warning about a large JavaScript chunk and a direct-eval warning originating from the ExcelJS dependency bundle. These are build observations, not runtime failure claims.

9. APPLICATION WALKTHROUGH - 1

the login form, Google sign-in option, navigation to registration/forgot password, and EDUVORA branding.

[SCREENSHOT REQUIRED: LOGIN PAGE]

Insert a full browser capture showing the login form, Google sign-in option, navigation to registration/forgot password, and EDUVORA branding.

Figure 2: LOGIN PAGE.

9. APPLICATION WALKTHROUGH - 2

the registration form with full name, email, mobile, password and referral-code fields.

[SCREENSHOT REQUIRED: USER REGISTRATION PAGE]

Insert a full browser capture showing the registration form with full name, email, mobile, password and referral-code fields.

Figure 3: USER REGISTRATION PAGE.

9. APPLICATION WALKTHROUGH - 3

the authenticated dashboard with active package cards and package-comparison/details interaction.

[SCREENSHOT REQUIRED: DASHBOARD AND PACKAGE CARDS]

Insert a full browser capture showing the authenticated dashboard with active package cards and package-comparison/details interaction.

Figure 4: DASHBOARD AND PACKAGE CARDS.

9. APPLICATION WALKTHROUGH - 4

the multi-section personal, contact, address and bank-information form, including field validation controls.

[SCREENSHOT REQUIRED: COMPLETE PROFILE PAGE]

Insert a full browser capture showing the multi-section personal, contact, address and bank-information form, including field validation controls.

Figure 5: COMPLETE PROFILE PAGE.

9. APPLICATION WALKTHROUGH - 5

profile header, account overview and profile/edit cards for the authenticated user.

[SCREENSHOT REQUIRED: MY PROFILE PAGE]

Insert a full browser capture showing profile header, account overview and profile/edit cards for the authenticated user.

Figure 6: MY PROFILE PAGE.

9. APPLICATION WALKTHROUGH - 6

referral link, generated QR-code card, referral statistics and referral table.

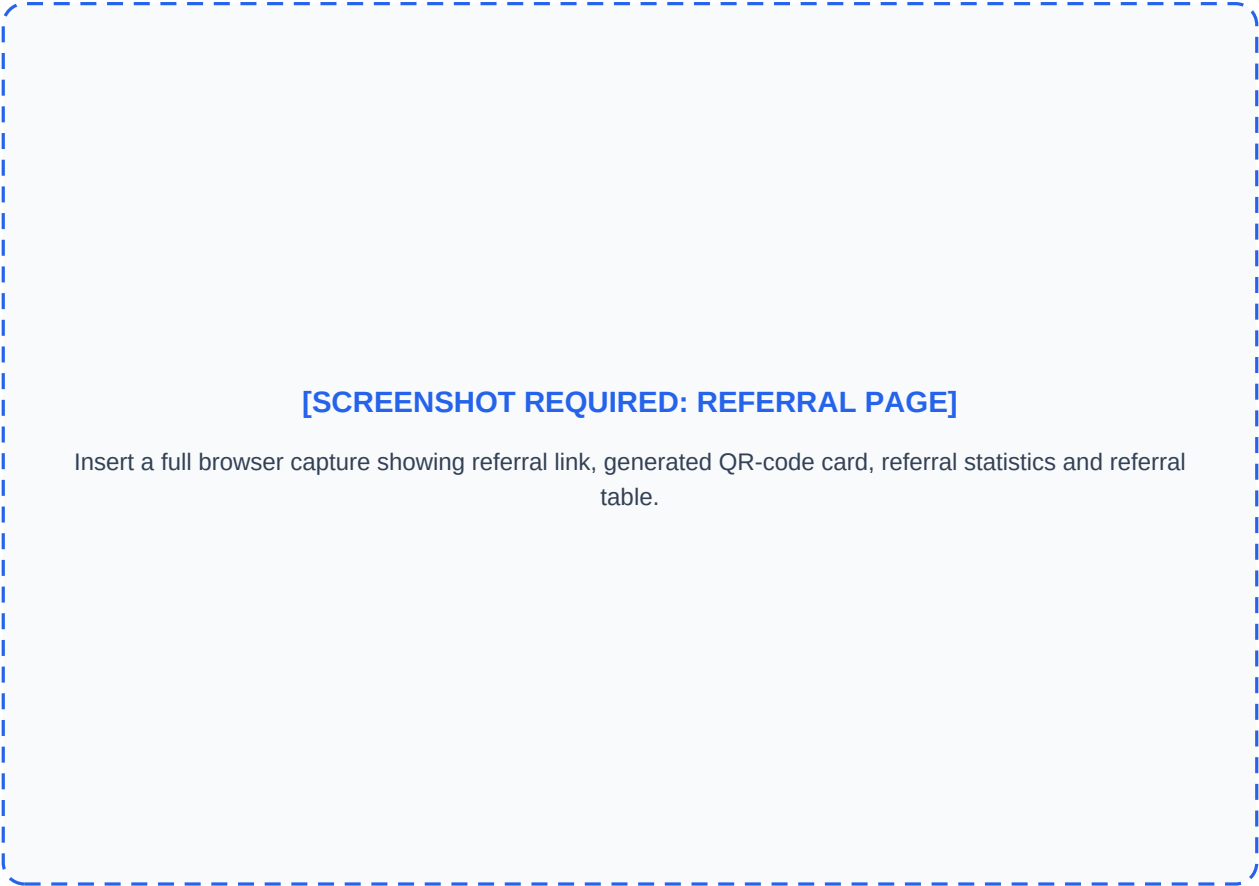


Figure 7: REFERRAL PAGE.

9. APPLICATION WALKTHROUGH - 7

team statistics, charts, tree search/zoom controls and the genealogy network tree.

[SCREENSHOT REQUIRED: MY NETWORK PAGE]

Insert a full browser capture showing team statistics, charts, tree search/zoom controls and the genealogy network tree.

Figure 8: MY NETWORK PAGE.

9. APPLICATION WALKTHROUGH - 8

a selected EDUVORA package and the payment action before Cashfree checkout.

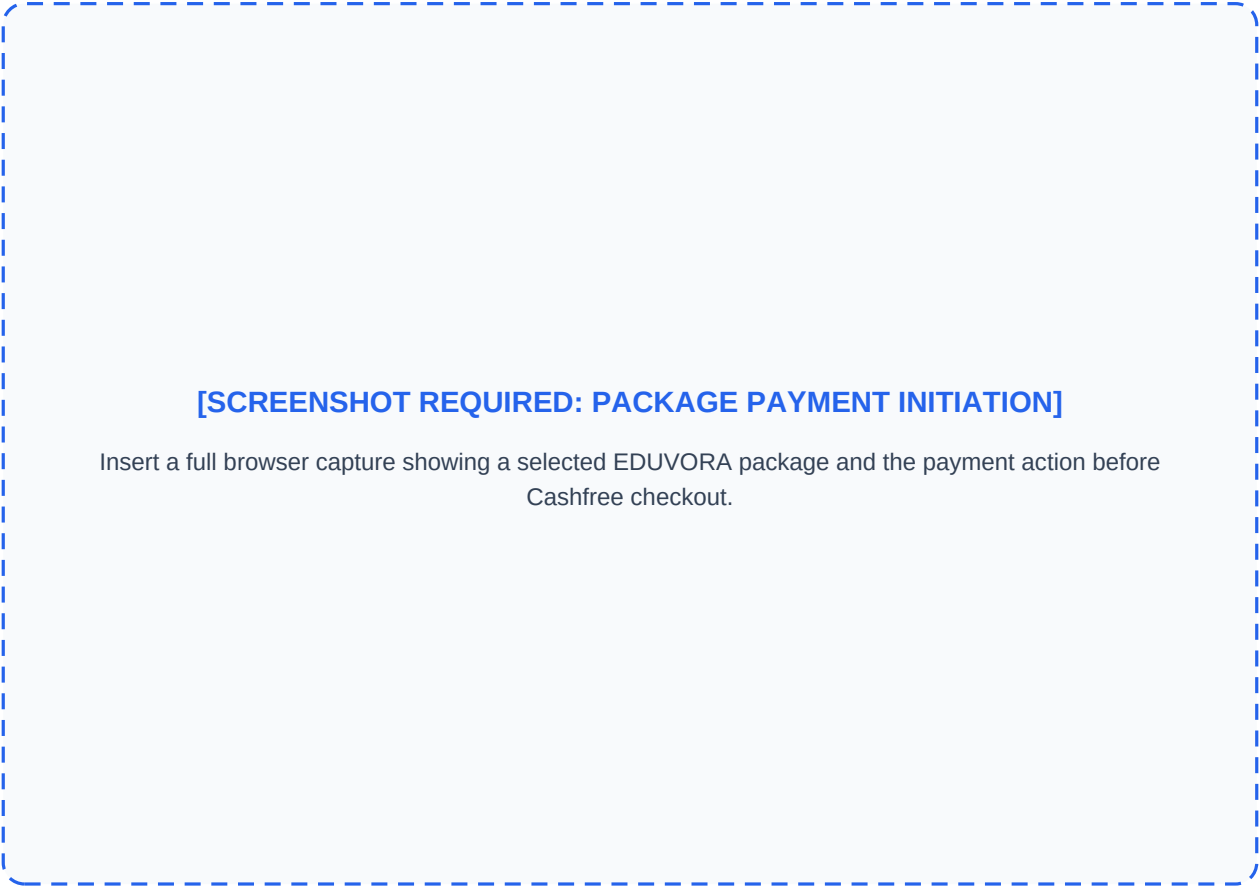


Figure 9: PACKAGE PAYMENT INITIATION.

9. APPLICATION WALKTHROUGH - 9

identity-proof selection, PAN and document fields, upload widgets, consent/review and KYC status.

[SCREENSHOT REQUIRED: KYC VERIFICATION PAGE]

Insert a full browser capture showing identity-proof selection, PAN and document fields, upload widgets, consent/review and KYC status.

Figure 10: KYC VERIFICATION PAGE.

9. APPLICATION WALKTHROUGH - 10

the current course-player layout with video/content area and expandable mock curriculum sidebar; label it as interface prototype data.

[SCREENSHOT REQUIRED: COURSE PLAYER PAGE]

Insert a full browser capture showing the current course-player layout with video/content area and expandable mock curriculum sidebar; label it as interface prototype data.

Figure 11: COURSE PLAYER PAGE.

9. APPLICATION WALKTHROUGH - 11

the current order table, filters and Excel export control; label the displayed rows as current frontend demo data.

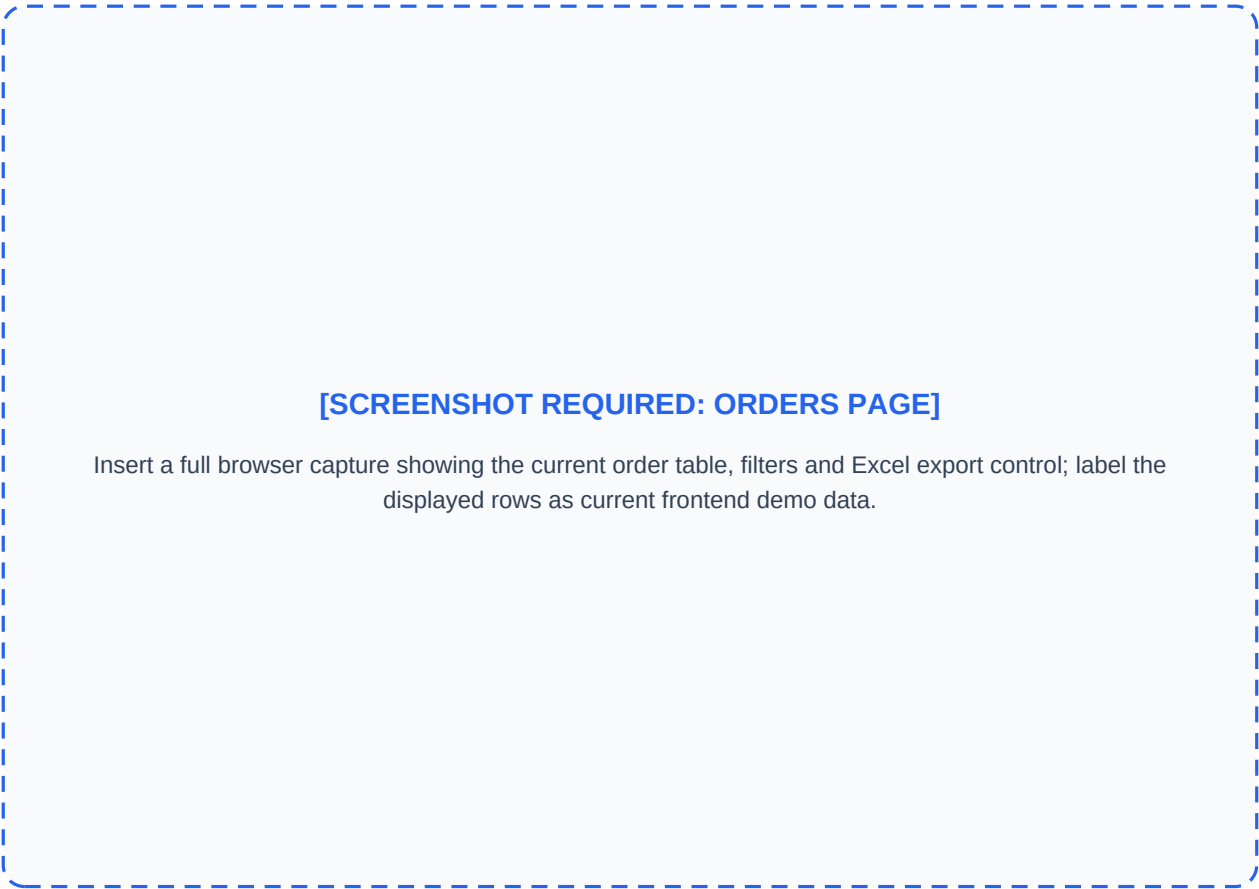


Figure 12: ORDERS PAGE.

9. APPLICATION WALKTHROUGH - 12

summary/history/wallet tabs and charts/cards; label this as current frontend presentation data unless a live backend source is added.

[SCREENSHOT REQUIRED: EARNINGS PAGE]

Insert a full browser capture showing summary/history/wallet tabs and charts/cards; label this as current frontend presentation data unless a live backend source is added.

Figure 13: EARNINGS PAGE.

10. CHALLENGES AND LIMITATIONS

10.1 Integration Boundaries

The system spans JavaScript and Python runtimes plus SQL Server, RabbitMQ, Cashfree, Google OAuth, SMTP and file repositories. The KYC workflow therefore depends on shared configuration and compatible path access between the Node API and Python worker. The Node publisher constructs a Celery protocol message through RabbitMQ's management HTTP API rather than an AMQP library, which is a deliberate cross-runtime integration detail requiring the RabbitMQ management endpoint.

10.2 Implementation Limitations

- Course playback and curriculum are currently mock data in CoursePlayer; no inspected API supplies enrolment/course content.
- Orders and earnings screens contain static/local display data; no matching backend order-history, wallet, commission or withdrawal endpoints are exposed.
- KYC accepts PDF MIME type but the Python quality checker uses `cv2.imread`, which cannot load a PDF as an image; a PDF may consequently fail worker quality processing.
- The worker callback URL is fixed to a localhost address in `tasks.py`, limiting deployment portability unless configuration is changed.
- KYC status logic has a potentially confusing interim state: submission procedure documentation says it sets `IsKYCVerified` while worker processing can still later reject the request.
- No automated tests, CI pipeline, deployment configuration or database migration scripts were found.

11. CONCLUSION

EDUVORA demonstrates a substantial web-development architecture centred on authenticated account workflows, SQL Server stored-procedure integration, referral relationships, package-payment initiation and an asynchronous identity-document pipeline. Its strongest technically verified contribution is the KYC flow: it separates immediate web-request handling from heavier document validation through RabbitMQ and Celery, adds image-quality checks and Aadhaar masking, and uses controlled temporary/permanent storage transitions.

The project also provides a broad and polished React interface for profile, referral, network, package, course, order and earnings experiences. A submission should distinguish this interface scope from the live backend scope: profile, referral, network, packages, payment initiation and KYC have visible route/controller implementations, while course curriculum, order history and earnings values remain presentation-stage data in the inspected codebase.

11.1 Future Enhancements

- Add secured live APIs and SQL procedures for purchased courses, course progress, orders, commissions, wallet transactions and withdrawals.
- Replace mock course/order/earnings data with protected server queries and apply user-level authorization.
- Verify Cashfree webhook signatures and authenticate the internal KYC callback.
- Make worker callback URL, OCR binary path and storage locations environment-configurable for deployment.
- Handle PDF KYC documents with an explicit rasterisation/extraction path or restrict worker-bound image uploads to supported formats.
- Add unit, integration and end-to-end tests, database migrations, CI and deployment documentation.
- Audit the super-admin route and enforce verified role checks on both client navigation and server APIs.

12. BIBLIOGRAPHY

Source	Use in this report
EDUVORA backend/brain.md	Database schema and stored-procedure documentation.
EDUVORA backend source	Routes, controllers, middleware, utilities, payment/KYC integrations and environment-variable categories.
EDUVORA frontend source	Routes, Redux state, screens, components, static-data boundaries and build configuration.
EDUVORA kyc-python-worker source	Celery task, RabbitMQ configuration, image quality, Aadhaar masking and SQL repository logic.
Package manifests and lockfiles	Verified frameworks and library dependencies.
Reference PDF: AI Career Prep Platform Report	Section order, academic tone, A4 report structure, certificate/abstract/acknowledgement/TOC pattern and screenshot convention.

Prepared for submission by Indrasish Biswas, Roll / Registration No. 24E102B91, B.Tech CSE.